

1. Memahami Serangan Buffer Overflow

a. Apa itu Buffer Overflow?

Buffer overflow adalah sebuah kondisi kerentanan keamanan di mana sebuah program mencoba menulis data ke dalam area penyimpanan sementara (*buffer*) di memori, tetapi jumlah data yang ditulis melebihi kapasitas yang dialokasikan untuk *buffer* tersebut. Akibatnya, data yang berlebih tersebut akan "meluber" dan menimpa area memori yang berdekatan. Bayangkan sebuah wadah yang hanya bisa menampung 1 liter air, tetapi kita mencoba menuangkan 2 liter air ke dalamnya. Sisanya pasti akan meluber keluar dari wadah.

Kerentanan ini paling sering ditemukan pada bahasa pemrograman seperti C dan C++, yang memberikan kontrol langsung atas memori dan tidak secara otomatis melakukan pemeriksaan batas (*bounds checking*) pada operasi penulisan data ke *buffer*. Fungsi-fungsi yang tidak aman seperti `strcpy()`, `strcat()`, `gets()`, atau `sprintf()` sering menjadi penyebab kerentanan ini karena mereka tidak memeriksa ukuran *buffer* tujuan sebelum menyalin data.

b. Dampak dan Konsekuensi

Pada awalnya, *buffer overflow* mungkin hanya menyebabkan program berhenti bekerja secara tidak terduga (*crash*) atau mengalami *segmentation fault*. Namun, bagi penyerang yang cerdas, *buffer overflow* adalah pintu gerbang menuju kendali penuh atas program dan bahkan sistem. Dengan menimpa area memori yang tepat, penyerang dapat:

- **Menyebabkan Penolakan Layanan (*Denial of Service - DoS*):** Membuat program atau sistem *crash*, sehingga tidak dapat diakses oleh pengguna sah.
- **Mengubah Alur Eksekusi Program:** Memanipulasi *pointer* atau alamat memori yang menentukan instruksi selanjutnya yang akan dijalankan program. Ini memungkinkan penyerang untuk "membelokkan" alur program ke kode berbahaya yang telah mereka sisipkan.
- **Eksekusi Kode Arbitrari (*Arbitrary Code Execution*):** Ini adalah skenario paling parah. Penyerang dapat menyuntikkan kode berbahaya (*malicious code*, sering disebut *shellcode*) ke dalam memori program yang rentan. Kemudian, dengan memanfaatkan *buffer overflow*, mereka mengubah alamat kembali (*return address*) dari sebuah fungsi di *stack* sehingga program akan melompat dan mengeksekusi *shellcode* mereka. Dengan *shellcode* ini, penyerang bisa mendapatkan akses *shell* pada sistem target, menginstal *malware*, mencuri data, atau melakukan tindakan lain yang merugikan.

2. Mekanisme Serangan Buffer Overflow (Pada Stack)

Serangan *buffer overflow* yang paling umum dieksploitasi adalah *stack-based buffer overflow*. Untuk memahami cara kerjanya, kita perlu sedikit memahami bagaimana memori komputer diatur, khususnya bagian yang disebut *stack*.

Stack adalah area memori yang digunakan program untuk menyimpan data sementara, seperti variabel lokal sebuah fungsi, parameter fungsi, dan yang paling penting, alamat kembali (*return address*). Alamat kembali ini adalah instruksi ke mana program harus "kembali" setelah sebuah fungsi selesai dijalankan.

Berikut adalah urutan kejadian dalam serangan *stack-based buffer overflow*:

1. **Fungsi Dipanggil:** Ketika sebuah fungsi dipanggil, sistem akan menempatkan *return address* (alamat instruksi setelah fungsi dipanggil) ke *stack*. Di atasnya, *frame pointer* fungsi sebelumnya akan disimpan, diikuti oleh variabel-variabel lokal dari fungsi yang baru dipanggil. Salah satu variabel lokal ini bisa jadi adalah *buffer* yang rentan.
2. **Input Berlebih:** Penyerang mengirimkan *input* (misalnya, melalui *form input* pengguna, parameter URL, atau *network packet*) yang ukurannya jauh melebihi kapasitas *buffer* yang dialokasikan dalam fungsi yang rentan.
3. **Data Meluber:** Karena tidak ada pemeriksaan batas, program akan terus menulis data ini melewati batas *buffer* yang dialokasikan. Data berlebih ini akan mulai menimpa data lain yang ada di *stack*, termasuk *frame pointer* dan, yang paling penting, *return address* yang disimpan di *stack*.
4. **Injeksi Kode Berbahaya (*Shellcode*):** Dalam *input* berlebih yang dikirim penyerang, mereka juga menyertakan *shellcode* (kode mesin kecil yang dirancang untuk melakukan tugas tertentu, seperti membuka *command shell*). *Shellcode* ini ditempatkan di suatu bagian dari *input* yang akan berakhir di memori program (misalnya, di dalam *buffer* itu sendiri atau di dekatnya).
5. **Penimpaan Return Address:** Penyerang menghitung dengan cermat berapa banyak data "sampah" yang harus dikirimkan sebelum *return address* di *stack*. Setelah data sampah ini, penyerang akan menempatkan alamat memori *shellcode* mereka. Jadi, *return address* yang asli akan tertimpa dengan alamat ke *shellcode* penyerang.
6. **Eksekusi Kode Penyerang:** Ketika fungsi yang rentan selesai dieksekusi, alih-alih kembali ke instruksi yang seharusnya (alamat kembali yang asli), program akan mengambil *return address* yang telah dimodifikasi oleh penyerang. Program akan melompat ke alamat ini, yang merupakan lokasi *shellcode* penyerang, dan mulai mengeksekusinya. Pada titik ini, penyerang telah berhasil mengendalikan alur eksekusi program.

Untuk meningkatkan peluang keberhasilan, penyerang sering menggunakan *NOP sled* (serangkaian instruksi "No Operation"). Ini adalah serangkaian instruksi yang tidak melakukan apa-apa selain menggeser *program counter* ke instruksi berikutnya. Penyerang menempatkan *NOP sled* sebelum *shellcode* mereka. Dengan demikian, *return address* tidak perlu menunjuk tepat ke awal *shellcode*, melainkan bisa menunjuk ke mana saja di dalam *NOP sled*. Program akan "meluncur" di atas *NOP sled* hingga mencapai *shellcode*.

3. Implementasi Serangan Buffer Overflow Sederhana (Konseptual)

Sebagai bagian dari tugas ini, kita akan membuat contoh program sederhana dalam bahasa C yang memiliki kerentanan *buffer overflow*. Kemudian, kita akan menjelaskan bagaimana penyerang bisa memanfaatkan kerentanan ini.

Peringatan: Program ini dibuat untuk tujuan edukasi dan demonstrasi. Jangan pernah mengimplementasikan kode yang rentan seperti ini di lingkungan produksi atau di sistem yang penting. Eksploitasi *buffer overflow* adalah tindakan ilegal jika dilakukan tanpa izin.

a. Kode Program C yang Rentan (vulnerable_program.c)

Berikut adalah contoh program C yang rentan terhadap *buffer overflow*. Program ini menggunakan fungsi `gets()`, yang sangat tidak aman karena tidak melakukan pemeriksaan batas.

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

// Fungsi untuk demonstrasi kerentanan

void vulnerable_function() {

    char buffer[64]; //Alokasi buffer berukuran 64 byte

    volatile int authenticated = 0; // Variabel boolean, volatile untuk mencegah optimasi kompiler

    printf("Masukkan nama Anda: ");

    gets(buffer); // Fungsi yang sangat rentan: tidak memeriksa batas input!

    if (authenticated) {

        printf("Autentikasi berhasil! Selamat datang, %s.\n", buffer);

        // Dalam skenario nyata, ini bisa memicu akses ke area sensitif atau

        shell system("/bin/sh"); // Contoh eksekusi shell (di Linux/Unix)

        // Di Windows bisa system("cmd.exe");

    } else {

        printf("Autentikasi gagal. Halo, %s.\n", buffer);

    }

}
```

Nama: Ariski Myardi
NIM: 1103220019

```
    }  
  
    printf("Selesai menjalankan fungsi.\n");  
}  
  
int main() {  
    printf("Program Buffer Overflow Sederhana Dimulai.\n");  
    vulnerable_function();  
    printf("Program Selesai.\n");  
    return 0;  
}
```

/*

Untuk mengompilasi program ini (di lingkungan Linux/GCC):

```
gcc -fno-stack-protector -z execstack -no-pie vulnerable_program.c -o  
vulnerable_program
```

-fno-stack-protector: Menonaktifkan stack smashing protection (fitur keamanan modern).

-z execstack: Mengizinkan stack untuk dapat dieksekusi (diperlukan untuk shellcode di stack).

-no-pie: Menonaktifkan Position Independent Executables (mempermudah prediksi alamat memori).

*/

Penjelasan Kerentanan pada Kode:

- **char buffer[64];**: Ini mendeklarasikan sebuah *buffer* bernama *buffer* dengan ukuran tetap 64 *byte*. Artinya, *buffer* ini hanya dirancang untuk menyimpan maksimal 63 karakter ditambah satu *null terminator* (`\0`).
- **volatile int authenticated = 0;**: Variabel *authenticated* ini dideklarasikan tepat setelah *buffer* di *stack*. Kata kunci *volatile* digunakan di sini untuk memastikan kompiler tidak mengoptimalkan variabel ini sehingga tetap berada di memori *stack* dan rentan untuk ditimpa. Variabel ini mengontrol apakah pesan sukses

Nama: Ariski Myardi
NIM: 1103220019

atau gagal ditampilkan, dan dalam skenario eksploitasi yang lebih kompleks, bisa mengontrol jalur kritis program.

- **gets(buffer);**: Inilah akar masalahnya. Fungsi gets() akan membaca *input* dari pengguna hingga ditemukan karakter *newline* (\n) atau akhir dari file (EOF), tanpa memedulikan seberapa besar buffer yang disediakan. Jika pengguna memasukkan lebih dari 63 karakter, gets() akan terus menulis karakter-karakter tersebut melewati batas buffer[64], menimpa variabel *authenticated* dan data-data lain di *stack* yang lokasinya lebih tinggi, termasuk *return address* fungsi *vulnerable_function()*.

Jika penyerang bisa menimpa variabel *authenticated* sehingga nilainya menjadi non-nol (misalnya 1), maka blok if (*authenticated*) akan tereksekusi.

4. Writeup Serangan Buffer Overflow

Untuk melakukan *writeup* serangan, kita akan berasumsi program *vulnerable_program.c* sudah dikompilasi menjadi *vulnerable_program* tanpa *stack protector* dan dengan *stack* yang dapat dieksekusi, seperti yang dijelaskan pada komentar kode di atas.

Tujuan Serangan: Mengubah nilai variabel *authenticated* dari 0 menjadi non-nol (misalnya 1), sehingga program akan mengeksekusi *system("/bin/sh")* dan memberikan *shell* kepada penyerang.

Lingkungan Asumsi:

- Sistem Operasi: Linux (atau lingkungan mirip Unix)
- Kompiler: GCC
- Arsitektur: x86 (32-bit)

Langkah-langkah Analisis dan Eksploitasi:

1. **Menganalisis Layout Memori (dengan Debugger Konseptual):** Kita bisa menggunakan *debugger* (misalnya GDB) untuk memahami tata letak *stack* pada saat *vulnerable_function()* dijalankan. Kita akan mencari tahu di mana variabel *authenticated* berada relatif terhadap *buffer*.

Dalam banyak kasus arsitektur x86 32-bit (tergantung kompiler dan versi GCC), variabel lokal yang dideklarasikan berurutan seringkali akan diletakkan berdekatan di *stack*. Karena *authenticated* dideklarasikan setelah *buffer*, kemungkinan besar ia akan berada tepat di atas (pada alamat memori yang lebih tinggi) dari *buffer*.

Misalkan *buffer* dimulai pada alamat 0xffffd000. Karena ukurannya 64 *byte*, *buffer* akan menempati alamat 0xffffd000 hingga 0xffffd03f. Variabel *authenticated* (tipe *int*, biasanya 4 *byte*) kemungkinan akan berada di 0xffffd040.

Jarak (offset) dari awal *buffer* ke variabel *authenticated* adalah 64 *byte*.

2. **Membangun Payload:** Untuk mengubah nilai *authenticated* menjadi 1, kita perlu mengirimkan *input* yang berisi:
 - 64 karakter *dummy* (sampah) untuk mengisi penuh *buffer*.

Nama: Ariski Myardi
NIM: 1103220019

- Kemudian, 4 *byte* yang merepresentasikan nilai 1 (misalnya, `\x01\x00\x00\x00` untuk sistem *little-endian*).

Payload (dalam bentuk string yang akan diketik/dilewatkan sebagai *input*): [64 karakter dummy] + [\x01\x00\x00\x00]

Contoh 64 karakter dummy:

[illegible][illegible]

Catatan: \x01\x00\x00\x00 adalah representasi biner dari angka 1 dalam format little-endian, yang umum digunakan pada arsitektur x86. Saat dimasukkan melalui keyboard langsung, ini mungkin tidak berfungsi karena karakter null (\x00) akan menghentikan string lebih awal. Namun, ini adalah konsep bagaimana exploit direpresentasikan secara biner. Untuk tujuan demonstrasi via keyboard, kita bisa menggunakan 64 'A' dan diikuti karakter apapun, lalu menekan enter. Ini akan menyebabkan crash jika tidak ada variabel authenticated yang bisa ditimpa, atau akan mengubah nilai jika variabel tersebut ada dan dapat ditimpa.

3. **Melakukan Serangan:** Kita akan menjalankan program dan memberikan *payload* yang telah dibuat.
4. # Contoh penggunaan command line untuk memberikan input panjang
5. # Di terminal:
6. # python -c 'print "A"*64 + "\x01\x00\x00\x00"' | ./vulnerable_program
7. # Atau, jika karakter non-ASCII bermasalah:
8. # printf
"AA
AAAAAAAAAAAAAAAAAAAA\x01" | ./vulnerable program

(Catatan: \x01\x00\x00\x00 sulit dimasukkan langsung melalui *keyboard*. Untuk simulasi sederhana, cukup mengirim 64 'A' dan satu karakter apa pun setelahnya, lalu menekan Enter. Jika program dirancang dengan baik, *authenticated* harusnya tertimpa.)

Hasil yang Diharapkan:

Ketika *payload* tersebut dimasukkan, *buffer* akan meluber dan menimpa variabel *authenticated* dengan nilai 1. Karena *authenticated* kini bernilai non-nol (true), kondisi *if (authenticated)* akan terpenuhi.

Nama: Ariski Myardi
NIM: 1103220019

Output yang akan terlihat:

Nama: Ariski Myardi
NIM: 1103220019

Program Buffer Overflow Sederhana Dimulai.

[illegible]

\$ # Ini adalah prompt shell baru yang terbuka (dari /bin/sh)

Pada titik ini, penyerang akan mendapatkan kendali atas *shell* sistem target, yang berarti mereka dapat menjalankan perintah apa pun yang diizinkan oleh hak akses program yang rentan tersebut.

Pentingnya Pengujian dan Pengamanan:

Contoh sederhana ini menunjukkan betapa mudahnya sebuah kerentanan *buffer overflow* dapat dieksploitasi jika tidak ditangani dengan benar. Oleh karena itu, dalam pengembangan perangkat lunak yang aman, sangat penting untuk:

- Menggunakan fungsi-fungsi aman (*safe functions*) yang melakukan pemeriksaan batas (misalnya `fgets()` daripada `gets()`, `strncpy()` daripada `strcpy()`).
- Mengaktifkan fitur keamanan kompiler seperti *stack protectors* (`-fstack-protector`).
- Menggunakan *Address Space Layout Randomization* (ASLR) untuk mempersulit prediksi alamat memori.
- Mengadopsi praktik pemrograman yang aman dan melakukan pengujian keamanan yang ketat, termasuk *fuzzing* dan *penetration testing*.

Kerentanan *buffer overflow* telah menjadi penyebab banyak serangan siber besar di masa lalu, menegaskan pentingnya perhatian terhadap perilaku program dan praktik pengamanan kode.